

An Application of Combined and Optimized File-Sharing Methods

Qizhen Sun, Jinbin Zhang, Yuhao Li, Yumeng He

Background

The concept of information transfer exists as a fundamental human quality, drawing together societies through access to new knowledge and furthering of communication. Historically, technologies such as the postal system, the telegraph, the telephone, and the radio emerged to reduce the obstacle of distance and accelerate social coordination. Now in the digital age and the Internet, the ability to communicate and transfer information has grown significantly. A primary function of the Internet serves to connect users together, providing methods of long-distance communication in real-time.

Initially, when files were primarily exchanged using removeable media such as CDs or USB drives, files were restricted to the size of the removeable media, forcing simplified formats and methods of compression. This warranted technical skill and knowledge for a drastically new technology. However, as the Internet grew larger and larger, the needs of user interaction and information exchange also grew. As a result, the nature of transferred content evolved dramatically, along with the skill of the average user. Modern digital communication involves large scale multimedia exchange, ranging from high-resolution photographs, 4K videos, and design assets to virtual machine images, software packages, and massive datasets [<https://link.springer.com/article/10.1007/s12243-014-0424-9>]. Reports on broadband traffic growth identify multimedia content and rich interactive applications as major drivers of long-term bandwidth demand [<https://link.springer.com/article/10.1186/s40649-015-0022-7>]. Therefore, file transfer has ceased to be only peripheral to the Internet economy, rather, it has now become one of its dominant functions.

Despite its necessity, the landscape of file transfer remains fragmented. Users today are faced with a confusing array of protocols and platforms, each with distinct limitations. For example, email is often hindered by attachment size limits and security vulnerabilities depending on its host. Cloud storage services offer convenience but rely on centralized servers, raising concerns regarding privacy, data ownership, and recurring subscription costs. Meanwhile, peer-to-peer (P2P) technologies offer direct transfer and high speed but often require a high level of technical configuration, alienating the average user.

This fragmentation not only reflects technical specialization, but also the capitalistic nature between platforms. Large technology companies design communication ecosystems to maximize user dependence rather than interconnectedness. Messaging platforms have evolved from simple communication utilities into multifunctional systems, integrating payments, media consumption, social networking, and cloud storage. Research on mobile messaging ecosystems shows that instant messaging platforms have increased function to central interfaces for digital life rather than merely tools for conversation [<https://pmc.ncbi.nlm.nih.gov/articles/PMC10031722/>]. The

result is a landscape in which users are pressured into adopting multiple applications with various add-on features, each optimized for maintaining their monopoly on certain features.

Privacy is also a main concern for users. Research on messaging applications demonstrates that even systems that advertise strong encryption frequently expose private information [https://www.researchgate.net/publication/363046548_Social_Media_Privacy_Issues_Threats_and_Risks]. Other studies examining messaging ecosystems show that many applications continuously communicate with hidden infrastructure, proposing surveillance and data-collection risks, especially with third-party hosted platforms [<https://journals.sagepub.com/doi/10.1177/21582440231181395>]. Consequently, users often sacrifice privacy and autonomy while attempting to perform normal tasks.

As data volumes continue to swell and remote collaboration becomes the standard rather than the exception, the efficiency of our digital interactions is increasingly dictated by the speed and reliability of these transfers. Consequently, there is a pressing need for a unified solution that abstracts away the complexity of underlying protocols.

Technical problems and challenges:

Developing a unified file transfer application requires navigating a complex landscape of networking and architectural challenges. Foremost among these is peer discovery, particularly within the constraints of the current IPv4 infrastructure. As most devices reside behind Network Address Translation (NAT), frequently behind multiple layers, establishing direct connections remains unstable, even for established protocols like BitTorrent [**SOURCE**]. Consequently, a 100% reliable peer-to-peer approach is currently unfeasible without a backup. To ensure connectivity, the proposed system must implement a centralized server transfer as a fallback when direct P2P discovery fails.

On a local level, LAN discovery presents hurdles specific to mobile hardware. Manufacturers often implement policies that limit local network scanning, either silently blocking discovery or requiring explicit user permission, which disrupts the seamless user experience. Furthermore, the global network environment poses distinct challenges, particularly regarding Deep Packet Inspection (DPI). In regions like China, ISPs actively inspect and throttle unknown encrypted UDP packets [**SOURCE**]. This creates a critical conflict between speed and survivability; while UDP-based protocols like QUIC offer superior transfer speeds, they are highly visible to DPI tools. Conversely, TCP-based transfers are slower due to congestion control but often more reliable across restricted networks. The application should therefore employ a flexible protocol design, potentially incorporating obfuscation strategies to ensure efficiency without triggering ISP blocks. It may take some extra effort to decide the most suitable protocol design.

Beyond connectivity, the architecture must also address the challenge of cross-platform development. While various ecosystems seem determined to remain incompatible with others, users who must use various products and platforms do not experience the same feelings

[SOURCE]. Implementing cross-platform development may require consolidating basic programming logic in one place such that it may be repeatedly used for each proposed interface.

Security and verification are equally paramount. Addressing users concerns regarding obfuscated communications and outsourced data-collection, the system should have a method of user verification to authenticate participants before a session begins, paired with end-to-end encryption using mature cryptographic libraries to prevent unauthorized access. End-transfer verification and integrity checks are essential components of the file transfer logic, as these mechanisms confirm that the data received matches the data sent.

Existing solutions and their drawbacks

Users today depend on multiple incompatible systems for different situations. The problem is not that existing systems are entirely ineffective. Many are highly optimized for their intended use cases. Rather, the problem is that nearly all existing solutions focus on only one transport pathway or operational model, with their own limitations. Consequently, users must constantly adapt their behavior to the limitations of each platform, instead of a unified system that selects the most efficient and secure transfer mechanism available.

Cloud storage

One major category of existing solutions consists of cloud-storage-based platforms. Services such as Google Drive, Dropbox, iCloud Drive, Baidu Netdisk, and PikPak all fundamentally rely on centralized storage infrastructure. In these systems, files are uploaded to remote servers and later downloaded by recipients. This model offers convenience, persistent storage, and synchronization across devices, but it introduces several important limitations.

Cloud-based transfer is bandwidth inefficient. A file must travel from the sender to the server and then from the server to the receiver, effectively doubling network usage. This is particularly inefficient when both users are physically close or on the same local network. For example, two devices connected to the same Wi-Fi network may still route transfers through distant cloud infrastructure instead of communicating directly. Large files therefore experience unnecessary latency, increased server costs, and slower transfer speeds.

Additionally, cloud-based systems depend heavily on centralized infrastructure. Users must trust service providers to maintain availability, security, and data integrity. Even when providers advertise encryption, metadata such as filenames, file sizes, timestamps, recipient relationships, and sharing patterns often remain visible to operators, a class of protocol-layer leakage that mirrors the metadata exposure characterized for end-to-end encrypted messaging [1].

Moreover, many cloud-storage systems monetize user dependence through subscription models, storage quotas, or ecosystem lock-in. Free users of cloud services often encounter restrictions on storage size, transfer frequency, and download speeds. Cloud-based systems are also poorly suited for transient or ad-hoc transfer. Users often do not want long-term cloud synchronization

merely to send one file to another person. In the end, many cloud platforms force users into excessively complex and expensive workflows unsuited for immediate, direct exchange.

Messaging applications

Messaging applications partially emerged as informal alternatives to traditional file-sharing systems. Platforms such as WeChat, WhatsApp, Telegram, and Discord are now widely used for quick file transfer because users already communicate through them daily. This convenience explains why many users send documents, videos, installation packages, or compressed archives through messaging systems instead of dedicated transfer platforms. Their ease of use and massive user bases make them attractive for minor interpersonal sharing.

However, messaging applications are fundamentally designed for communication. Consequently, they impose numerous technical and operational limitations, with file-size restrictions making large file transfers near impossible. Some systems also aggressively compress images or videos, reducing quality. Others limit upload frequency, retention duration, or available cloud storage. Server upload bandwidth may also become a bottleneck, especially in regions with poor international connectivity. [SOURCE] Furthermore, messaging applications rarely provide advanced transfer semantics such as resumable chunk verification, transport migration, or optimized peer-to-peer local transfer.

In many cases, these platforms also centralize user dependence. Files remain stored on provider-controlled servers, and users become tied to specific ecosystems because their contacts already exist there. With these messaging platforms consolidating into their own multi-service ecosystems, the same issues with security and architectural costs prevail [].

Private client-server transfer

Technical users may opt to use private client-server transfers. For example, tools such as OpenSSH, SCP, rsync, and FTP-based systems are widely respected for reliability and efficiency. These tools provide secure encrypted communication and powerful synchronization mechanisms.

However, these tools generally assume one possesses significant technical knowledge. Users must often configure servers, manage authentication keys, selectively expose ports through routers or firewalls, and understand various other networking concepts. In many cases, users must also manage dynamic DNS services or port forwarding on home routers [SOURCE]. Such workflows are inaccessible to many non-technical users. Maintaining a self-hosted server can also introduce security risks and administrative overhead. Consequently, while these systems are powerful and efficient, they remain inaccessible to many ordinary users who simply require a reliable and secure way to transfer files without extensive setup procedures.

Server-based distribution

Large-scale server-based file distribution systems, including HTTP download servers and CDNs, are highly effective for distributing public content such as software packages, videos, or web

assets. However, they are not intended for private interpersonal transfer. Instead, these systems optimize one-to-many distribution rather than secure peer exchange.

Furthermore, self-hosted personal systems do not scale well. Residential Internet connections struggle with maintaining public accessibility due to NAT traversal problems, carrier-grade NAT (CGN) restrictions, dynamic IP allocation, and ISP limitations. CGN in particular is now deployed across thousands of autonomous systems, with mobile-operator networks among the most heavily affected []. From the perspective of service providers, large-scale centralized distribution systems such as HTTP servers, FTP infrastructure, or CDNs require significant operational investment and continuous maintenance. As a result, service providers incur bandwidth and infrastructure limitations.

Peer-to-peer

Peer-to-peer systems historically attempted to address some of these issues by enabling direct decentralized sharing between users. Research on decentralized sharing networks such as BitTorrent demonstrates the enormous scale and social importance of peer-based distribution systems []. However, traditional peer-to-peer systems were designed primarily for public distribution rather than private interpersonal transfer. In BitTorrent-style systems, content becomes discoverable to participants within the swarm. Once files are uploaded into the network, any participant with the appropriate metadata may potentially download them. This model is highly effective for large-scale public distribution but poorly aligned with private secure exchange. Attempts to mediate this with cryptographic extensions to peer-to-peer file sharing, for example, attribute-based encryption for fine-grained access control [], remain at research-prototype maturity rather than consumer deployment.

Traditional peer-to-peer systems also suffered from several additional problems. First, they frequently involved inconsistent or weak security models. Early systems often prioritized distribution efficiency over authentication or confidentiality. Second, usability was poor. Users commonly needed to manage torrent files, trackers, magnet links, port forwarding, and seeding behavior. Third, mobile support historically remained limited because many peer-to-peer protocols were designed around desktop networking assumptions. Fourth, peer-to-peer systems became culturally associated with copyright infringement and piracy, generating legal and reputational concerns. [SOURCE]

LAN-based transfer

Applications such as LANDrop are optimized for high-speed local-network transfer. Similarly, gaming platforms such as Steam now support LAN-based game-transfer functionality to reduce redundant Internet bandwidth consumption. These systems can achieve excellent performance because they transfer data directly across local infrastructure rather than through remote servers.

However, LAN-based systems become ineffective once users leave the same network environment. Different subnets, enterprise Wi-Fi isolation, hotel networks, carrier restrictions, or

remote geographic locations frequently prevent operation entirely. Many LAN-transfer applications also rely on local discovery protocols that are unreliable across heterogeneous network environments or heavily restricted mobile operating systems.

Bluetooth-assisted transfer

Bluetooth-based transfer systems suffer from a different set of limitations. Technologies such as AirDrop combine Bluetooth-based discovery with higher-speed Wi-Fi transfer mechanisms. These systems work effectively for nearby devices, yet remain heavily ecosystem-dependent. In particular, AirDrop functions exceptionally well within the Apple ecosystem, but offers no interoperability with Windows or Android environments. Additionally, AirDrop contact-discovery handshake itself has been shown to leak identifying information, including phone numbers and email addresses, to a passive on-network attacker, bringing privacy into question again [1].

Internet-based peer-to-peer applications

Some applications attempt Internet-based peer-to-peer transfer, such as Send Anywhere. While these tools improve interoperability compared with ecosystem-specific systems, they still generally rely on narrow transport assumptions or centralized relay infrastructure. They often lack robust offline fallback, advanced resumability, or unified multi-transport behavior.

PAKE-based short-code transfer

A small but architecturally significant category consists of tools that establish an end-to-end-encrypted channel between two peers using only a short shared code, such as Magic Wormhole and croc. These tools use the SPAKE2 password-authenticated key exchange protocol [2] to derive a strong session key from a low-entropy code (typically four to six human-pronounceable words), in a manner that resists offline attack: a passive observer of the relay must complete an online interaction per candidate code, and a failed attempt invalidates the session. The relay server holds no cryptographic access to the file contents. This cryptographic model is mature and is the closest existing design to the system we propose. The gap, however, is at the consumer-facing layer: existing implementations are distributed as command-line tools or basic web front-ends, lack first-class mobile clients, do not implement a LAN-direct fast path, do not support resume on failure, and centralize all traffic through a single default relay.

Furthermore, the network conditions actually encountered by real users vary substantially across regional deployments, with significant heterogeneity in achievable QUIC and UDP throughput and connection survival across regions [3], a dimension that none of the surveyed tools explicitly addresses.

The core problem is that none of the existing applications succeeds in putting all secure file transfer approaches into one single coherent application available across the major end-user platforms. Existing systems typically optimize for exactly one transport pathway. Very few

systems attempt to combine these approaches into a unified architecture that adapts to various situations. Consequently, users must manually choose different applications depending on many different constraints. Users encounter inconsistent interfaces, false encryption guarantees, and incompatible workflows.

Proposed solution

To address the fragmentation identified in current file transfer ecosystems, we propose a unified secure file-transfer platform that integrates multiple transport pathways into a single, cohesive application. By consolidating disparate solutions—such as local Wi-Fi Direct, peer-to-peer relays, and cloud-based storage—into one interface, this system drastically lowers the barrier to entry for users. No longer required to manually select a transfer method based on file size or network topology, the app simplifies use by determining the proper method on its own. For non-tech users, they can simply scan and transfer without knowing what approach has actually been used.

The architecture is designed to be both robust and highly customizable. While the default experience prioritizes ease of use, the platform will provide advanced options for users to deploy and configure their own relay servers. This flexibility empowers users to maintain control over their data, mitigating reliance on centralized third-party infrastructure and addressing concerns regarding data sovereignty. The core technical innovation, however, lies in the application's adaptive decision-making engine. Instead of adhering to a rigid protocol, the system will feature an intelligent transport selector that automatically determines the most appropriate algorithm for the specific network environment. All transfer approaches would be tried in parallel, and among successful ones, the one with highest priority will be selected.

This engine will analyze real-time network conditions (latency, packet loss, presence of firewalls or Deep Packet Inspection) to dynamically switch between transmission modes. For instance, in a restricted network environment where UDP packets are throttled, the system will automatically prioritize a TCP-based or obfuscated connection to ensure delivery; conversely, in an open local network, it will leverage high-speed UDP protocols like QUIC to maximize throughput. Ideally, an obfuscation strategy would be applied according to the network environment. Usually, the more aggressive the obfuscation is, the lower the transfer speed will be. Our algorithm aims to maintain the maximum possible speed while with minimum obfuscation that allows the packets to survive whatever kind of inspection.

A unified secure file-transfer platform therefore represents more than a technical optimization. It addresses a broader infrastructural problem. Such a system could reduce dependence on centralized intermediaries, improve interoperability across platforms, simplify secure communication for non-technical users, and provide adaptive transport behavior across diverse network environments. By consolidating multiple transfer paradigms into a single coherent application, such a platform would reduce friction, improve efficiency, and restore simplicity to one of the Internet's most fundamental functions: the exchange of information.

Papers (to be cited)

Blockchain based p2p <https://doi.org/10.1016/j.future.2022.11.010>

Also blockchain <https://doi.org/10.3390/s22145100>

Hole-punching with QUIC <https://doi.org/10.48550/arXiv.2408.01791>

Secure cloud <https://doi.org/10.48550/arXiv.2305.18639>

Bandwidth distribution across countries <https://doi.org/10.1186/s40649-015-0022-7>

Broadband traffic drivers, multimedia growth <https://doi.org/10.1007/s12243-014-0424-9>

Super-app ecosystem security SoK <https://arxiv.org/abs/2306.07495>

Metadata leakage in E2EE messaging (Session) <https://arxiv.org/abs/2002.04609>

QUIC performance in heterogeneous networks <https://doi.org/10.1002/nem.70022>

P2P communication across NATs (Ford et al.) <https://bford.info/pub/net/p2pnat/>

Carrier-grade NAT deployment measurement <https://doi.org/10.1145/2987443.2987474>

BitTorrent measurement and analysis https://doi.org/10.1007/11558989_19

Attribute-based encryption for P2P sharing <https://doi.org/10.1080/24751839.2021.1941574>

AirDrop privacy attack and PrivateDrop

<https://www.usenix.org/conference/usenixsecurity21/presentation/heinrich>

SPAKE2 password-authenticated key exchange (RFC 9382)

<https://datatracker.ietf.org/doc/rfc9382/>

Magic Wormhole secure file transfer <https://magic-wormhole.readthedocs.io/>